



CSCI/ECEN 3302

Introduction to Robotics

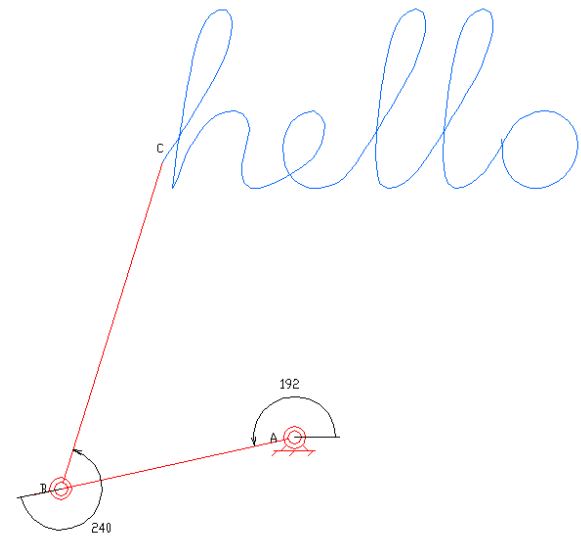
Alessandro Roncone

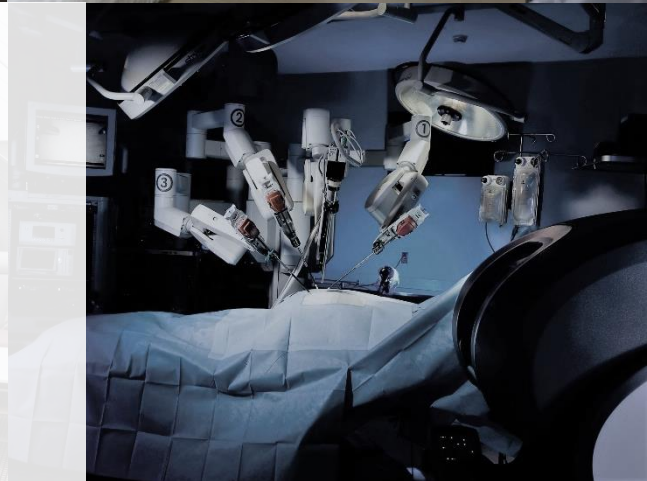
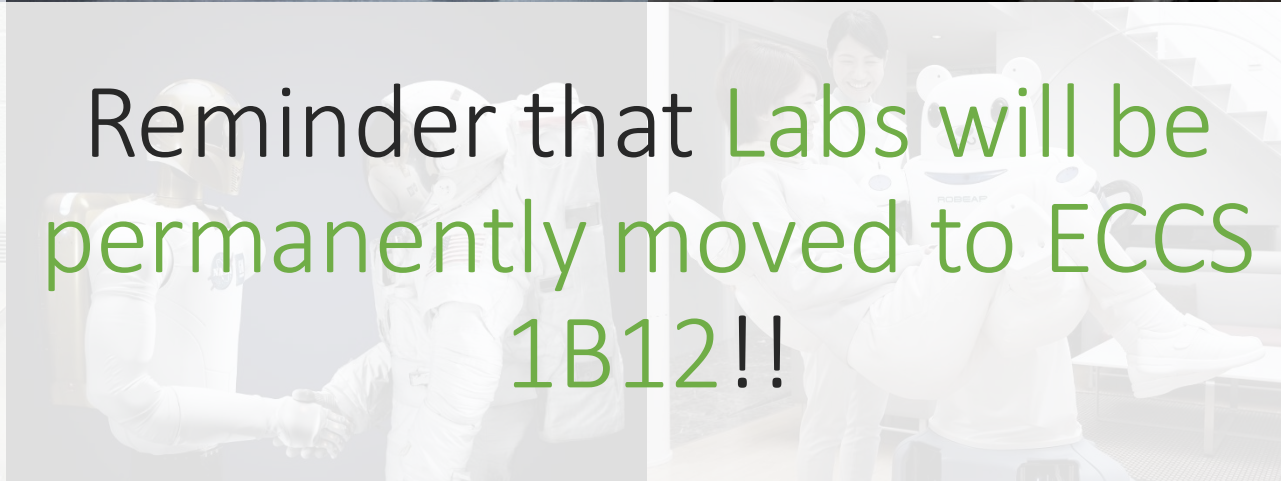
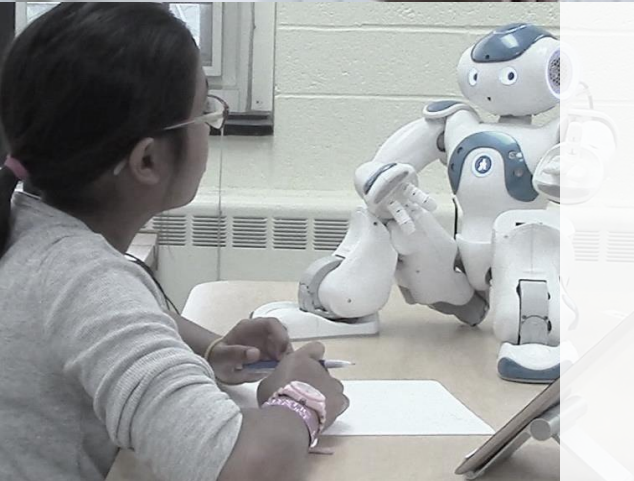
aroncone@colorado.edu

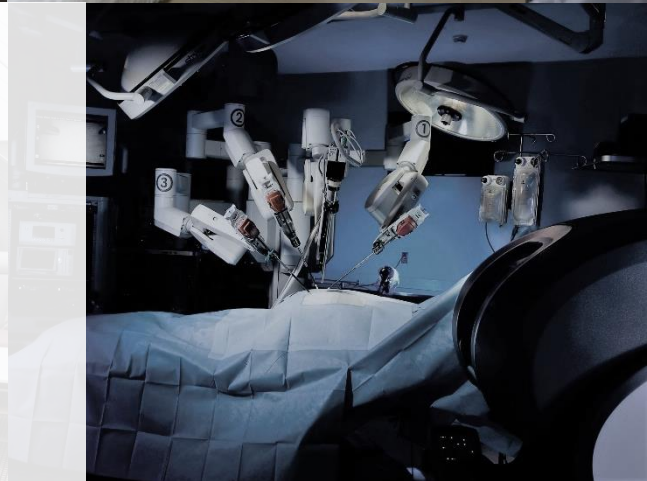
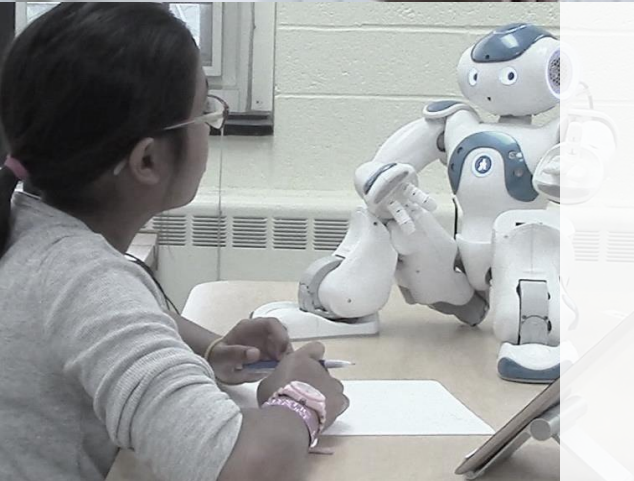
<https://hiro-group.ronc.one>

Administrivia

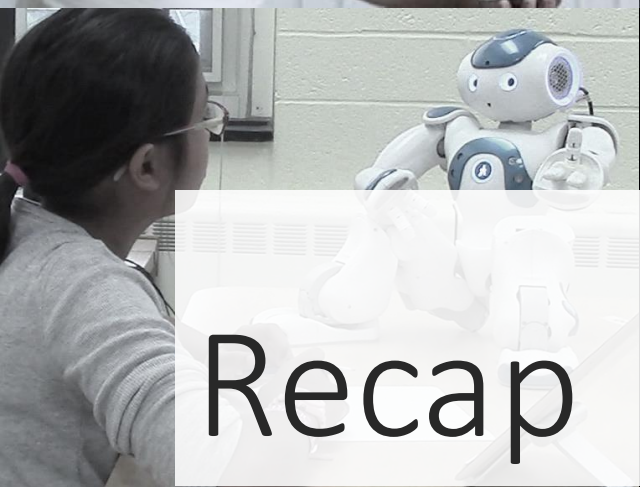
- Lab 3 due tonight!
- HW2 out today, due in two weeks
- Lab 4 out tomorrow, due in one week
 - NOTE: Lab 4 will be 1-week long!
 - Why? Because it's not hard 😊





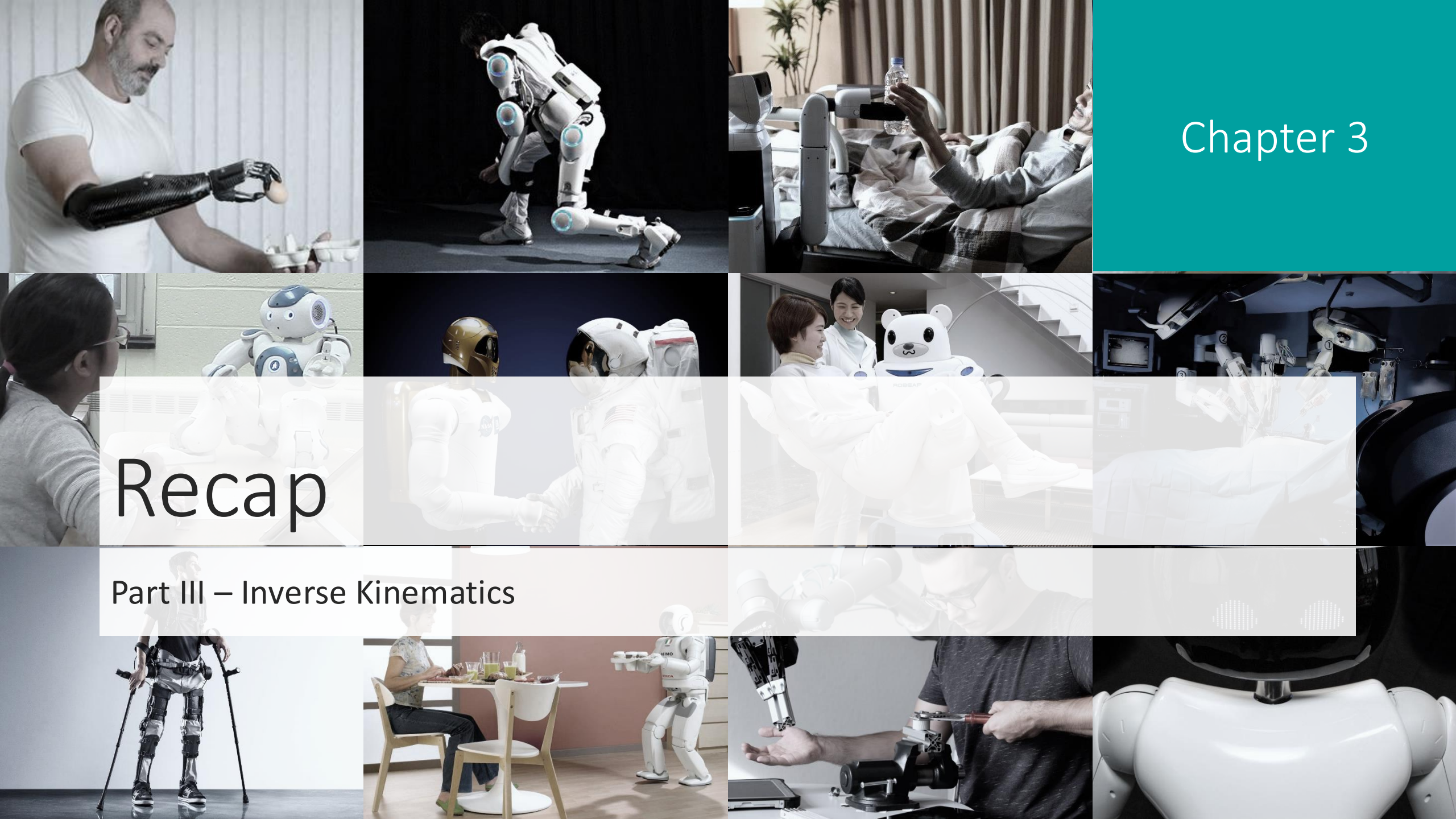


Chapter 3

A woman with dark hair and glasses is seen from the side, looking towards a small, white and blue humanoid robot sitting on a white table. The robot has a friendly face with large eyes and a small mouth. The background is a plain, light-colored wall. The word "Recap" is overlaid in large, black, sans-serif font across the bottom half of the image.

Recap

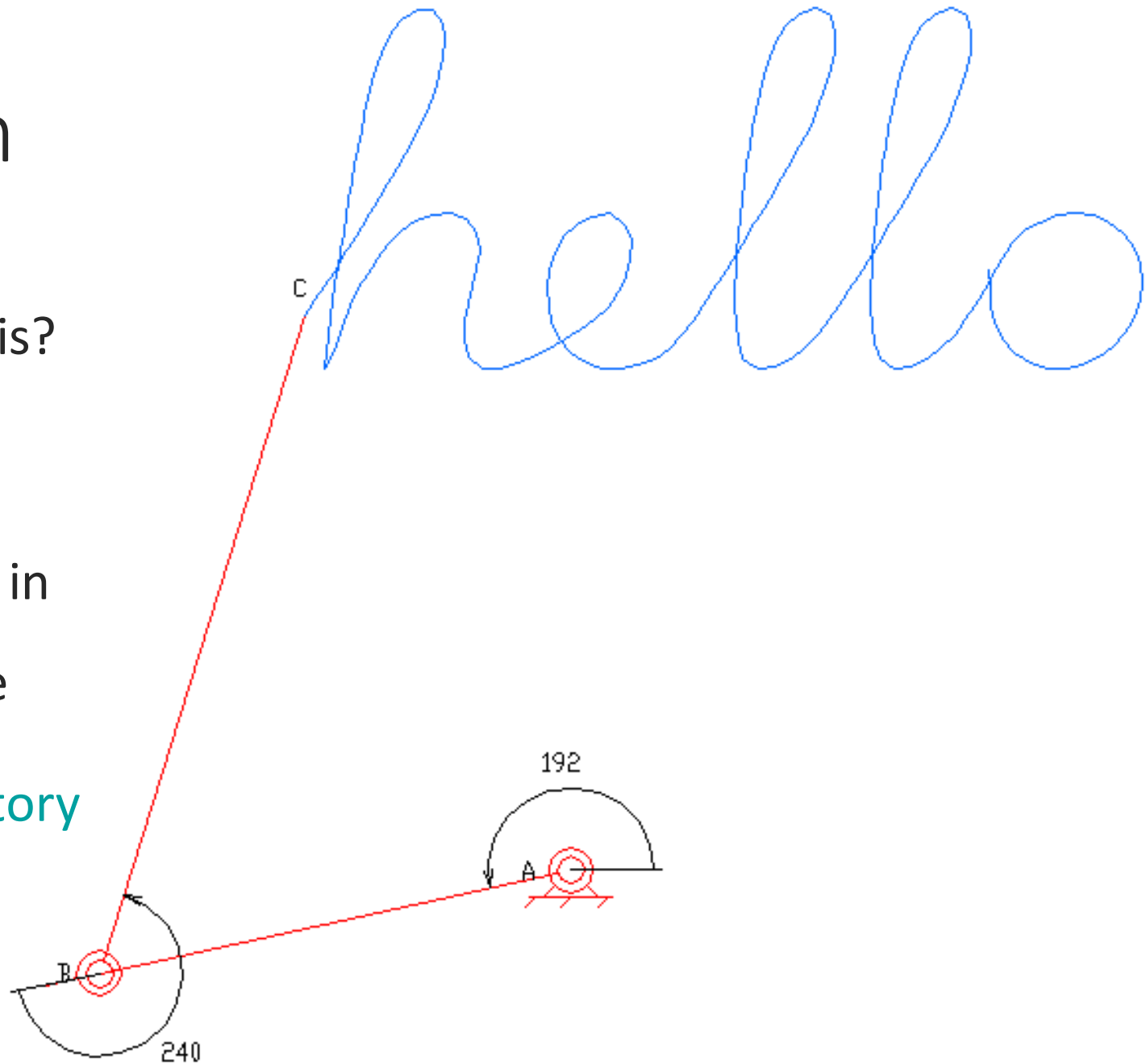
Part III – Inverse Kinem



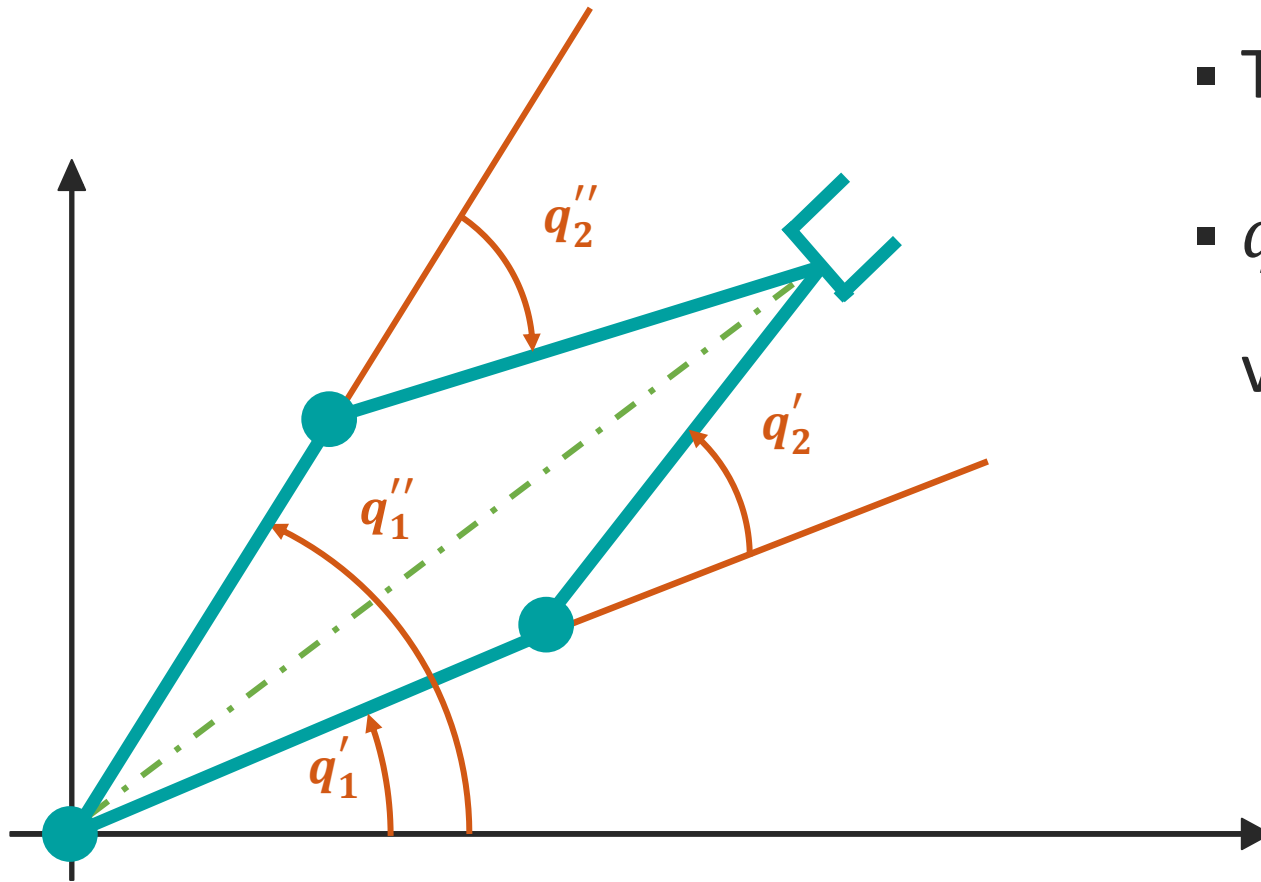
Motivating Problem

How can we get a robot to do this?

- We need to **define a trajectory** in end-effector/operational space
- We need to **convert this trajectory** to joint/configuration space



Example: Inverse Kinematics of a 2R arm



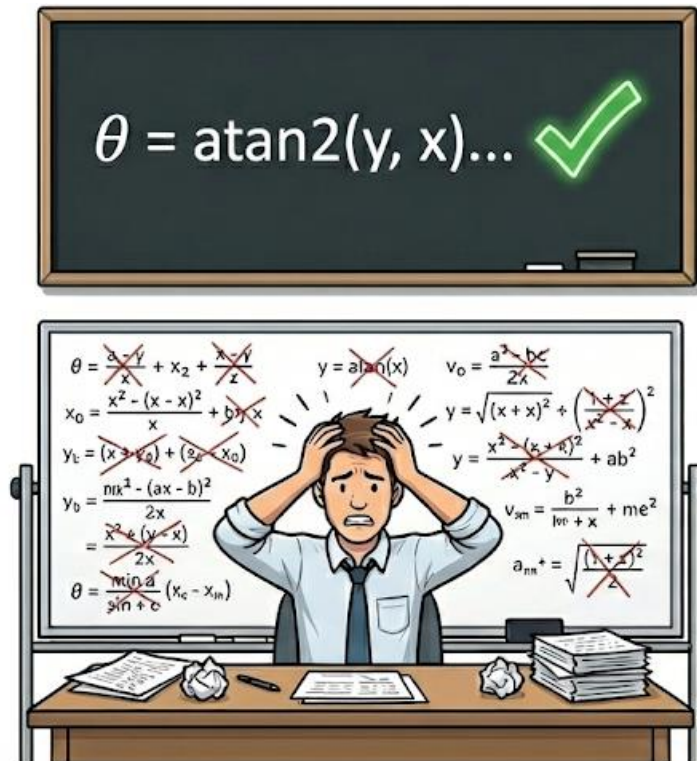
- Two solutions!
- q_2' and q_2'' have same absolute value, but different sign

Great video about IK:

<https://www.youtube.com/watch?v=IKOGwoJ2HLk>

Analytical solutions

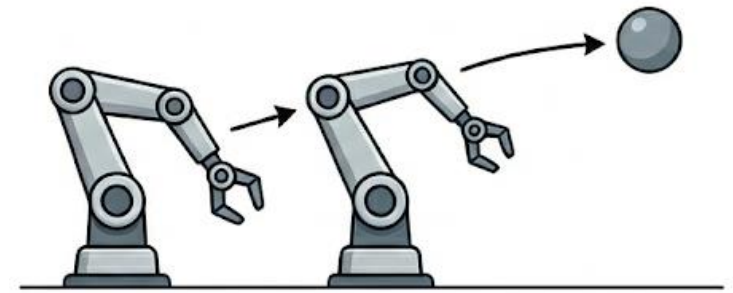
- Mathematically correct
- Code-efficient (once found)
- Extremely difficult derivation
- Robot-specific



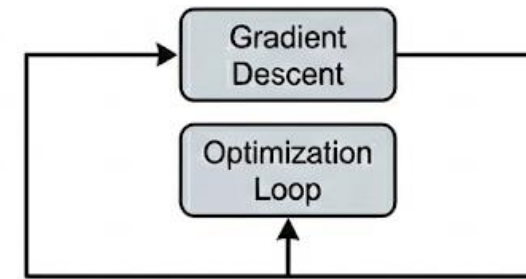
Numerical Solutions

[A.k.a. Optimization-based]

Iterative, "Best-Effort" Optimization

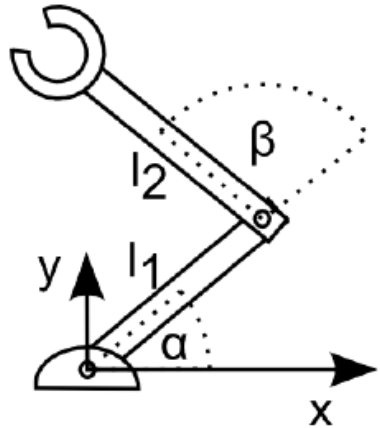


Iteration 1 Iteration 2 Convergence...



- General purpose algorithms
- Minimal domain knowledge
- Scalable to complex robots
- Approximate but practical
- Implicitly solve the full position control problem

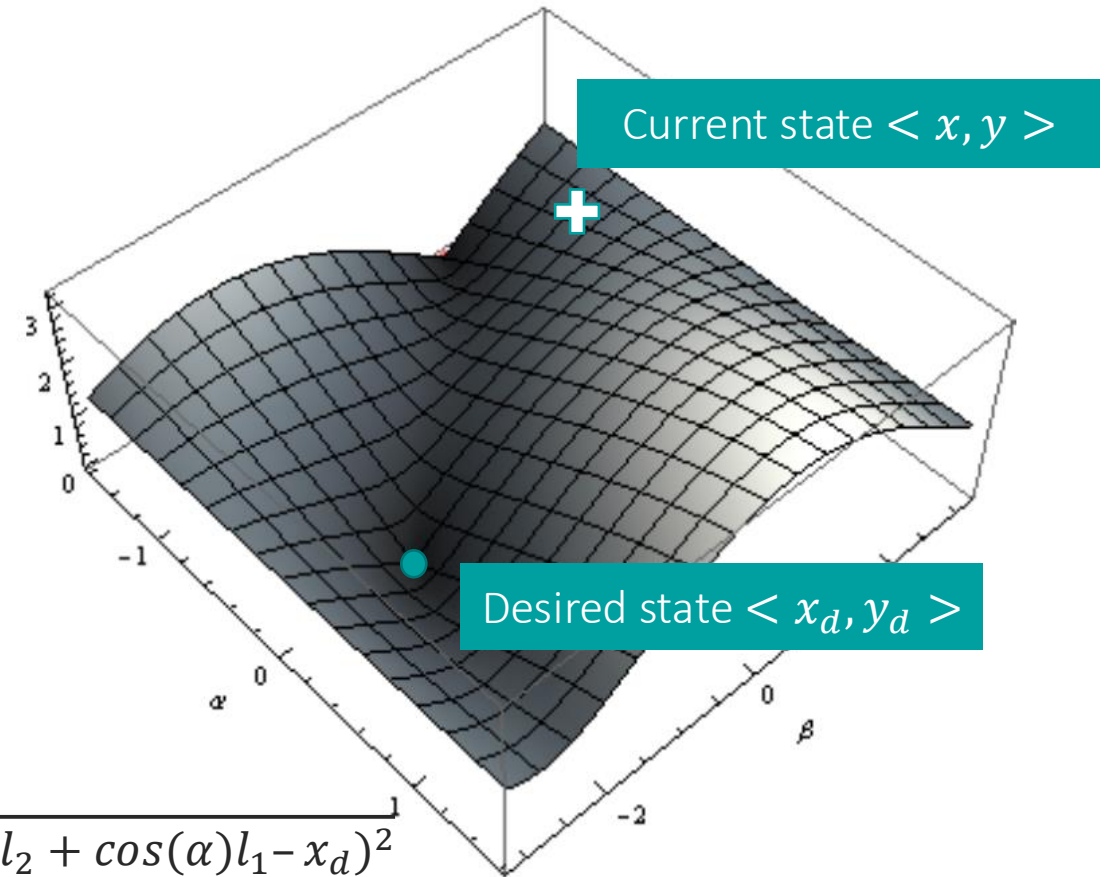
Easier ways to solve the IK problem



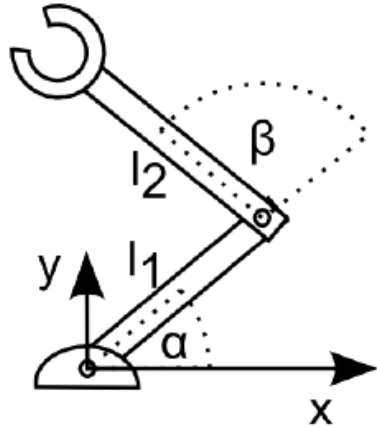
$$\begin{aligned}x &= l_1 \cos(\alpha) + l_2 \cos(\alpha + \beta) \\y &= l_1 \sin(\alpha) + l_2 \sin(\alpha + \beta)\end{aligned}$$

Just the Euclidean distance
between two vectors!

$$f_{x,y}(\alpha, \beta) = \sqrt{(\sin(\alpha + \beta)l_2 + \sin(\alpha)l_1 - y_d)^2 + (\cos(\alpha + \beta)l_2 + \cos(\alpha)l_1 - x_d)^2}$$



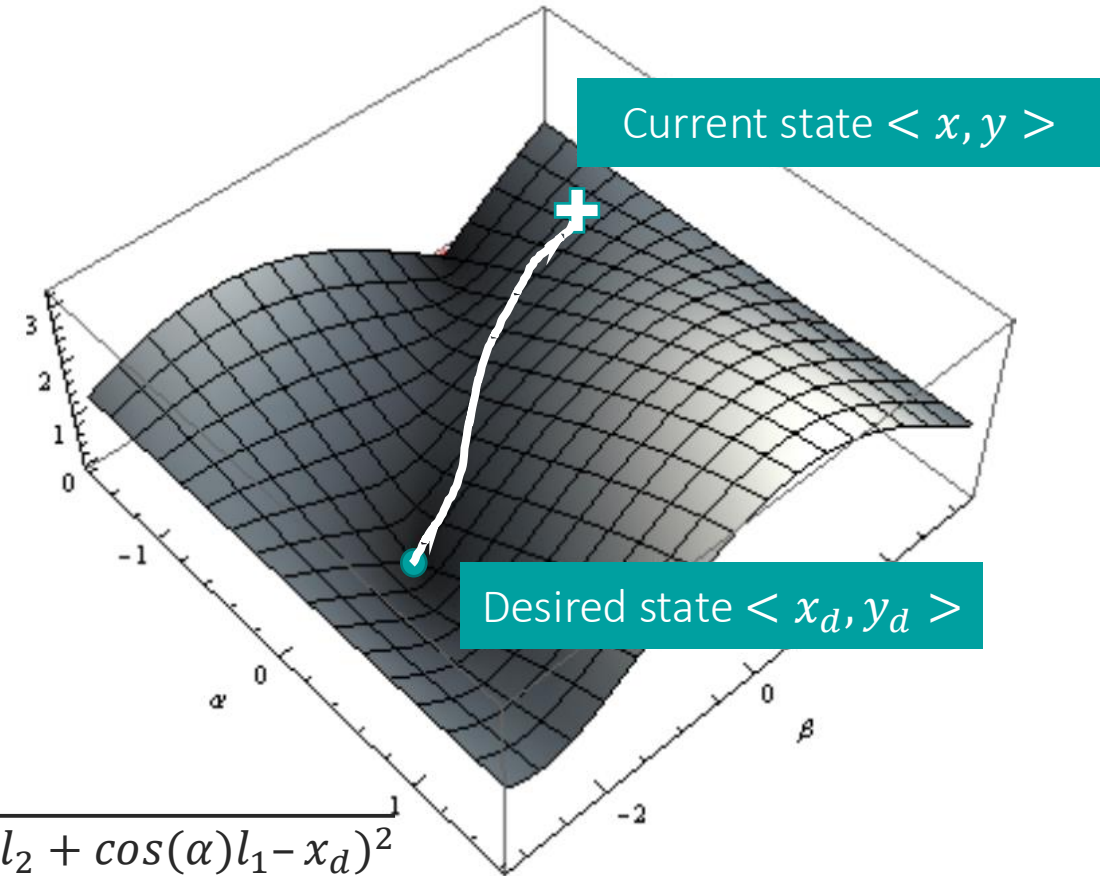
Motion Planning in EE Space



$$\begin{aligned}x &= l_1 \cos(\alpha) + l_2 \cos(\alpha + \beta) \\y &= l_1 \sin(\alpha) + l_2 \sin(\alpha + \beta)\end{aligned}$$

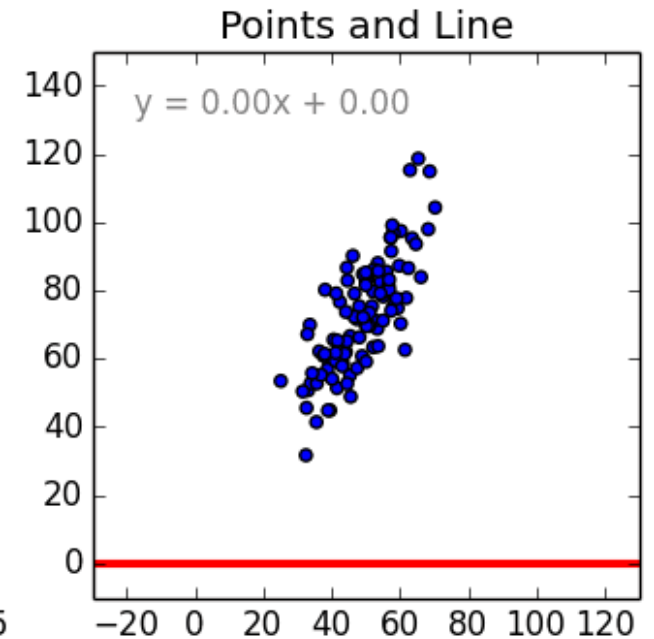
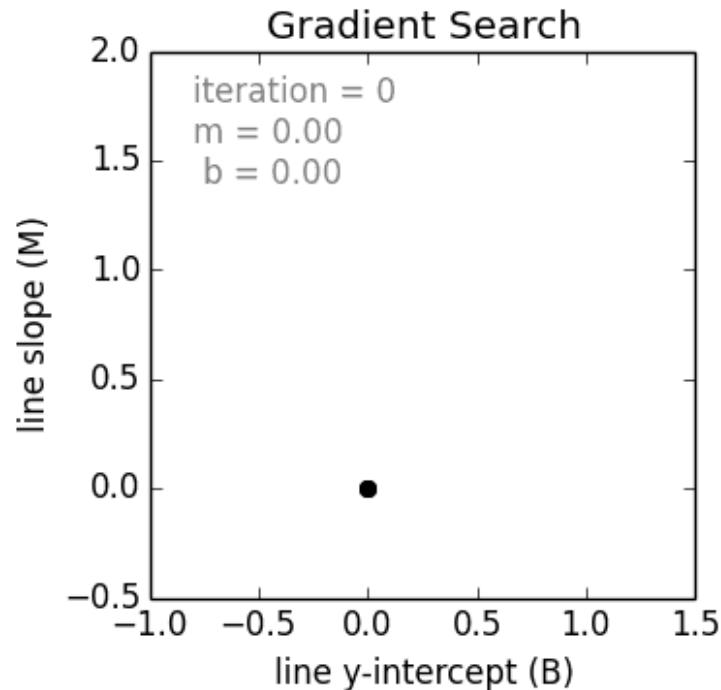
Just the Euclidean distance
between two vectors!

$$f_{x,y}(\alpha, \beta) = \sqrt{(\sin(\alpha + \beta)l_2 + \sin(\alpha)l_1 - y_d)^2 + (\cos(\alpha + \beta)l_2 + \cos(\alpha)l_1 - x_d)^2}$$



Gradient Descent

- Avoids pitfalls of univariate optimization
- Requires a gradient (can be analytic or empirical)
- Takes steps that optimize across all variables



How do we Move the Robot?

- Linear equations dictate end-effector position:

$$x_e(\alpha, \beta) = l_1 \cos(\alpha) + l_2 \cos(\alpha + \beta)$$

$$y_e(\alpha, \beta) = l_1 \sin(\alpha) + l_2 \sin(\alpha + \beta)$$

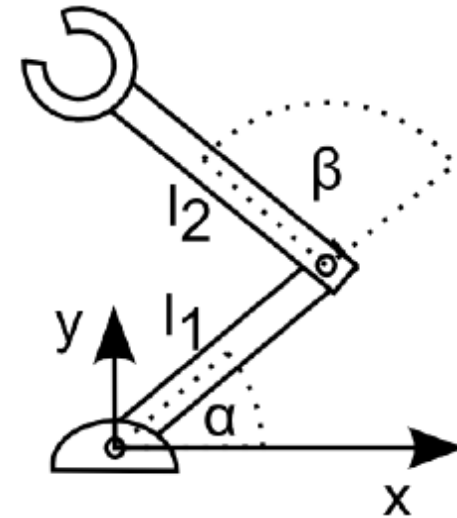
Forward Kinematics Equations

- Relationship between position change and angle change:

$$\delta x_e = \frac{\delta x_e(\alpha, \beta)}{\delta \alpha} \Delta \alpha + \frac{\delta x_e(\alpha, \beta)}{\delta \beta} \Delta \beta$$

$$\delta y_e = \frac{\delta y_e(\alpha, \beta)}{\delta \alpha} \Delta \alpha + \frac{\delta y_e(\alpha, \beta)}{\delta \beta} \Delta \beta$$

- $J = \begin{bmatrix} \frac{\delta x_e}{\delta \alpha} & \frac{\delta x_e}{\delta \beta} \\ \frac{\delta y_e}{\delta \alpha} & \frac{\delta y_e}{\delta \beta} \end{bmatrix}$ Change in Position = $J \cdot \dot{q}$



x_e : x position of end effector

y_e : y position of end effector

q : Robot pose in C-space

$\dot{q}$: Change in C-space

Using the Jacobian to Move the Robot

- $\frac{dp_e}{dt} = J \frac{dq}{dt}$, or in other words , $v_e = J \cdot \dot{q}$

- $\dot{q} = J^{-1} \cdot [v_{e,d} + K(p_{e,d} - p)]$

- With a single equation, you are automatically performing gradient descent!

$\dot{q}$: Change in C-space

K: gain

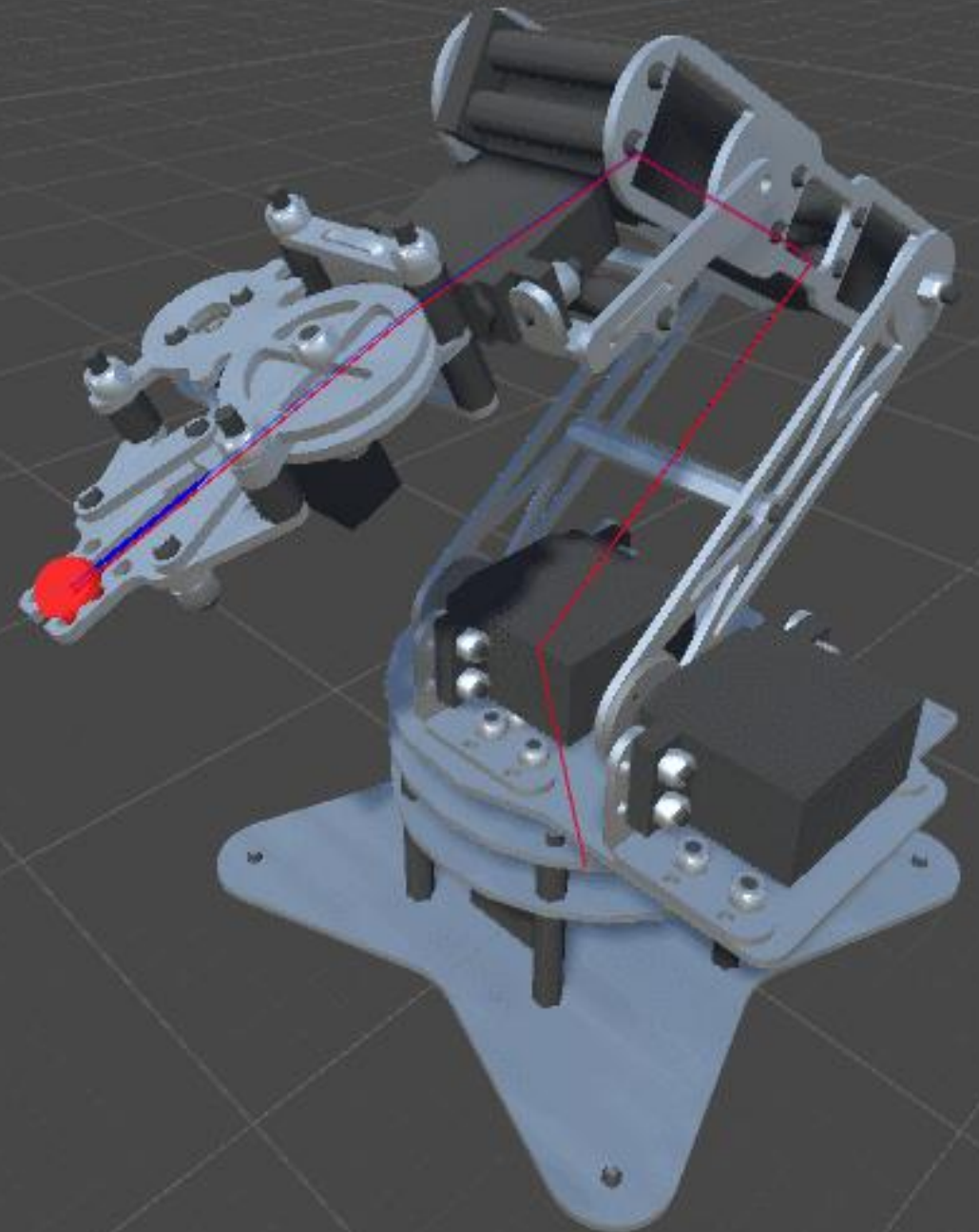
$v_{e,d}$: Desired velocity

$p_{e,d}$: Desired position

Convergence Problems!

$$\nabla f_{\alpha_0} = (\alpha_0, \alpha_1, \alpha_2) = \frac{f(\alpha_0 + \Delta x, \alpha_1, \alpha_2) - f(\alpha_0, \alpha_1, \alpha_2)}{\Delta x}$$

$$\alpha_0 \leftarrow \alpha_0 - L \nabla f_{\alpha_0}(\alpha_0, \alpha_1, \alpha_2)$$



Joint Angle

Limitation Problems!

How do we fix this?



Fast IK Planning



1x

Recap : Kinematics and its abstractions

	“Static” Kinematics (positions)		Differential Kinematics (velocities)	
	Forward Kinematics	Inverse Kinematics	Forward Kinematics	Inverse Kinematics
Holonomic Robots 	Possible and useful Lecture 3!	<u>Analytical solutions</u> Hard to compute / not scalable	Not really interesting / useful	<u>Numerical solutions</u> / Optimization Lecture 5! HW2!
Non-holonomic Robots 	Possible but not useful	Possible but not useful	Possible and useful Lab 2 / Lecture 4!	<u>Analytical solutions</u> Possible and useful Lab 3!

Where do we go from here?



6:16:34 05/06/2015

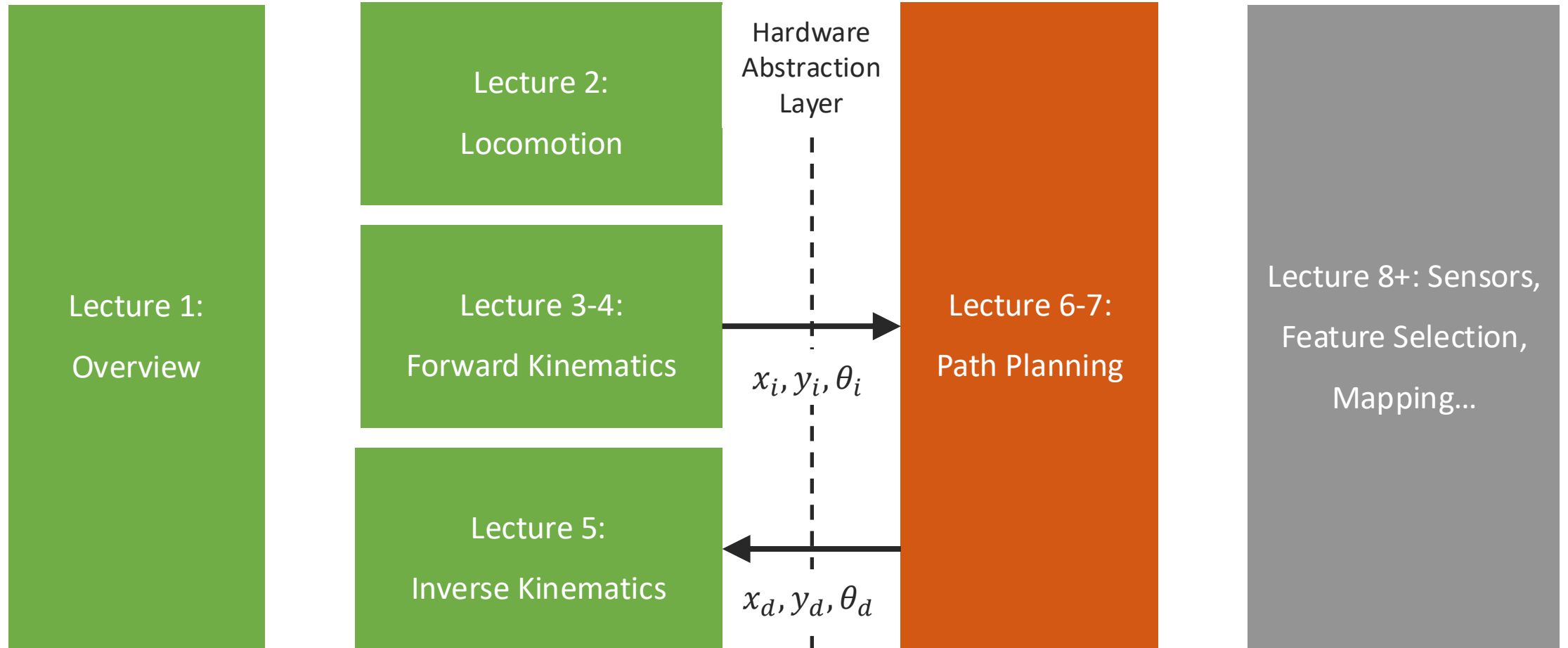
Chapter 4

Module 2 – COMPUTATION

Part I – Motion and Trajectory Planning



Roadmap



Motion Planning



Motion Planning

Some taxonomy

- **Kinematics**: **geometrical** relationships in terms of position/velocity between joint-space and task-space
- **Dynamics**: relationships between joint torques and **dynamical properties** of the plant with links/end-effector motions
- **Control**: computation of the **control actions** (i.e. joint torques) to achieve a desired motion.
- **Planning**: planning of the **desired** movements of the manipulator

Motion Planning

$\xi \rightarrow$ “xi”

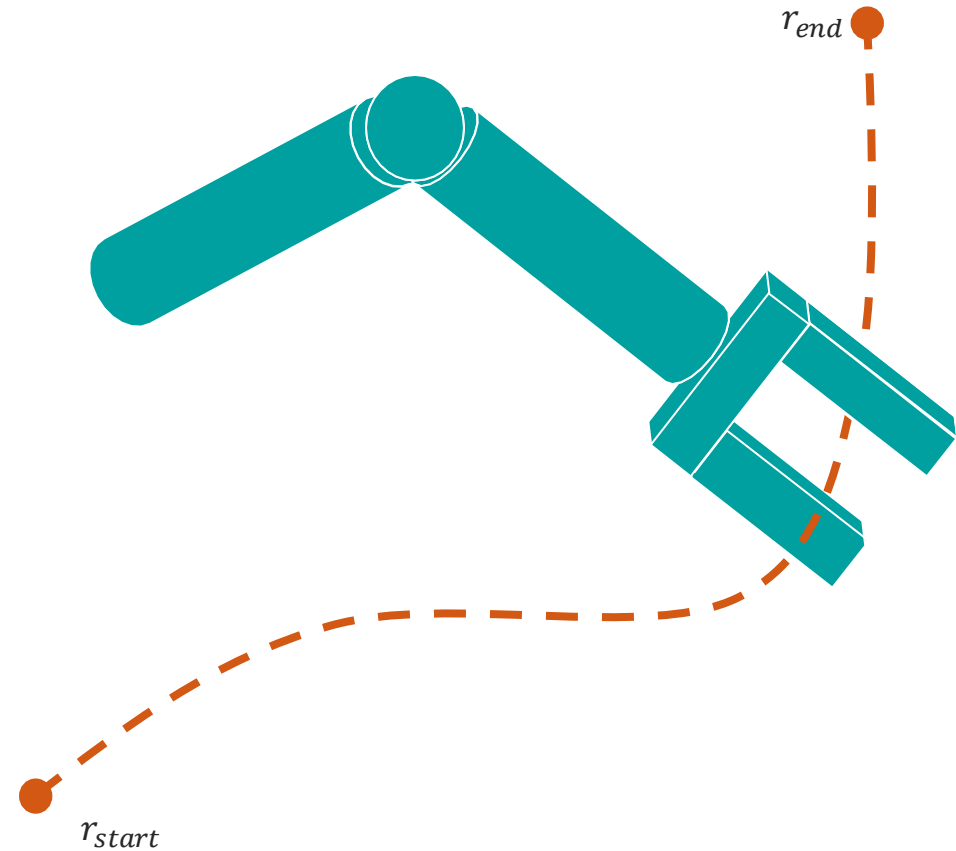
$\Xi \rightarrow$ capital “xi”

Both pronounced “ksee”

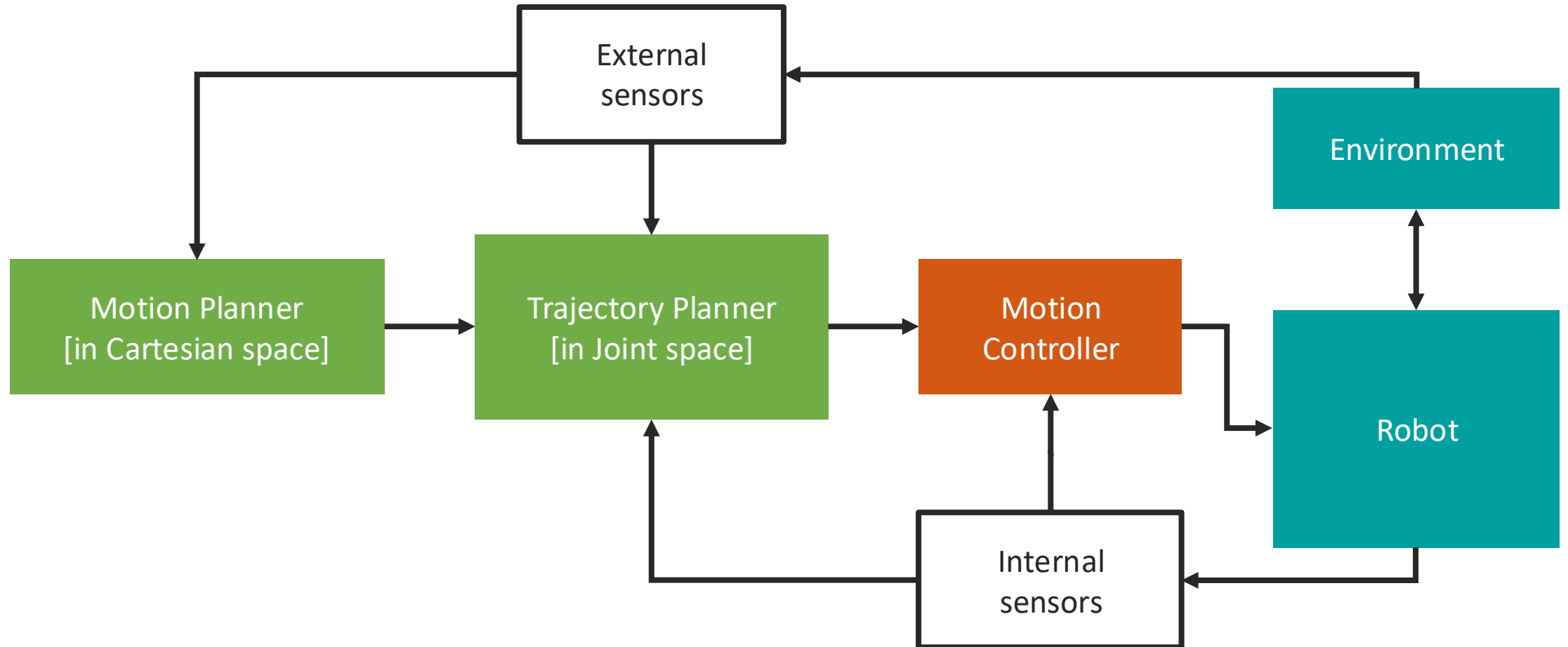
- **Goal: Find a trajectory** in configuration space from one point to another
 - A trajectory is usually denoted as $\xi: t \in [0, T] \rightarrow \mathcal{C}$
 - *(A function mapping time to robot configurations)*
 - The set of all trajectories is Ξ
- The Bad News:
 - All complete search algorithms scale exponentially!!
 - Motion planning problems are high dimensional, e.g., big exponent.

What is a trajectory?

- **Path**: geometric description of a series of poses from r_{start} to r_{end} that respect specifications (e.g. avoiding obstacles or following a specific force profile)
- **Trajectory**: execution of path over time $r(t)$



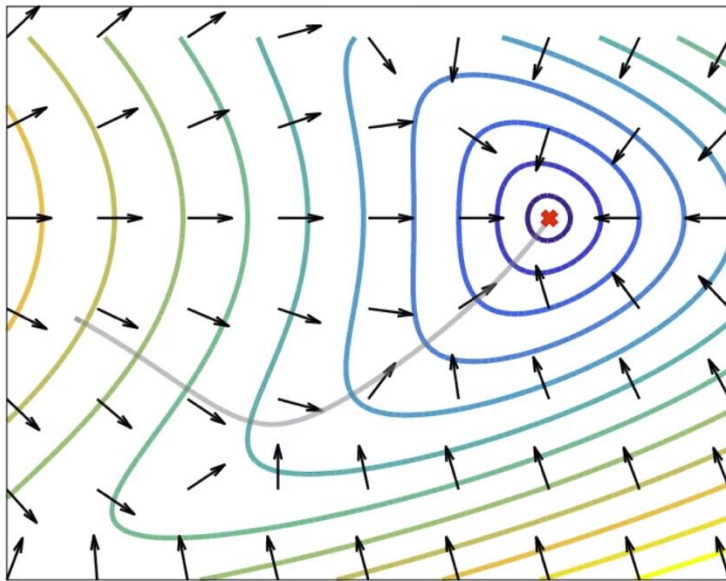
Trajectory Planning Interfaces



Optimal Control

AKA: Finding the best control policy for a desired goal

Closed-Loop Solutions

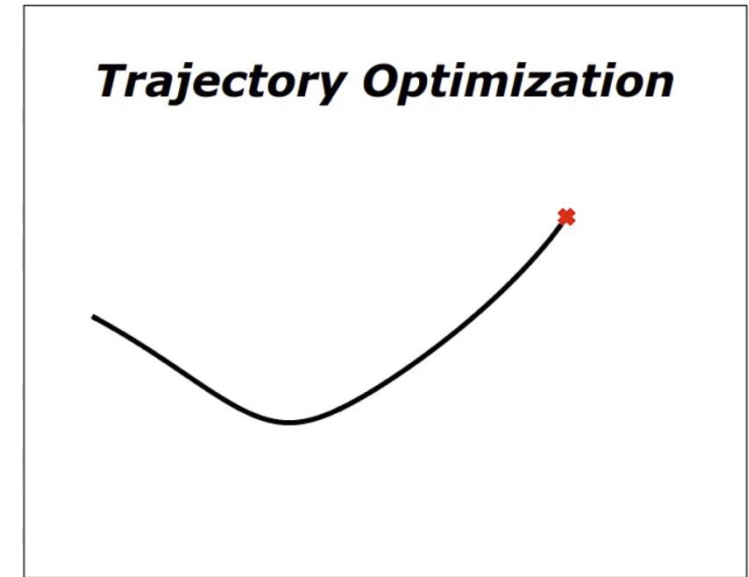


$$u = u(x)$$

“Global Method”: Gives action at all states
Generally, very expensive to compute

u designates
control signal

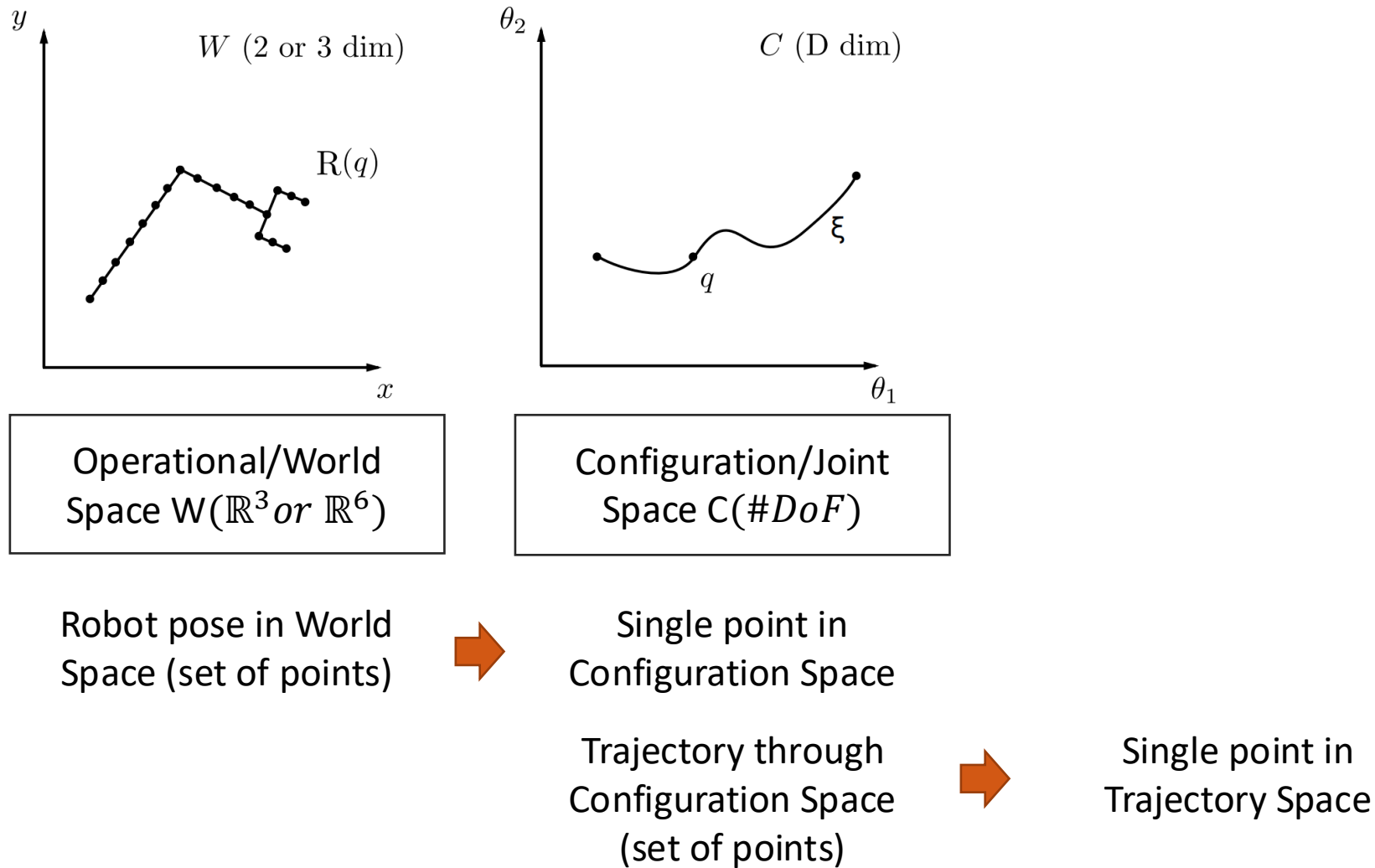
Open-Loop Solution



$$u = u(t)$$

“Local Method”: Gives action at relevant states
Usable in high dimensions

Problem Specification: Spaces



Trajectory Optimization

Problem Statement

- Trajectory $\xi: t \in [0, T] \rightarrow \mathcal{C}$ Maps time to configurations
- Objective Function $U: \Xi \rightarrow \mathbb{R}^+$ Maps trajectories to scalars
- The objective U encodes traits we want our path to have
 - Path length
 - Efficiency
 - Obstacle avoidance
 - Legibility
 - Uncertainty reduction
 - Human comfort
- Goal: Leverage the benefits of randomized sampling with asymptotic optimality

Set of possible trajectories



Our goal: Use a feedback controller that minimizes path length for our low dimensional problem

Problem Specification: Optimization

Trajectory Optimization seeks to find an optimal trajectory ξ^* :

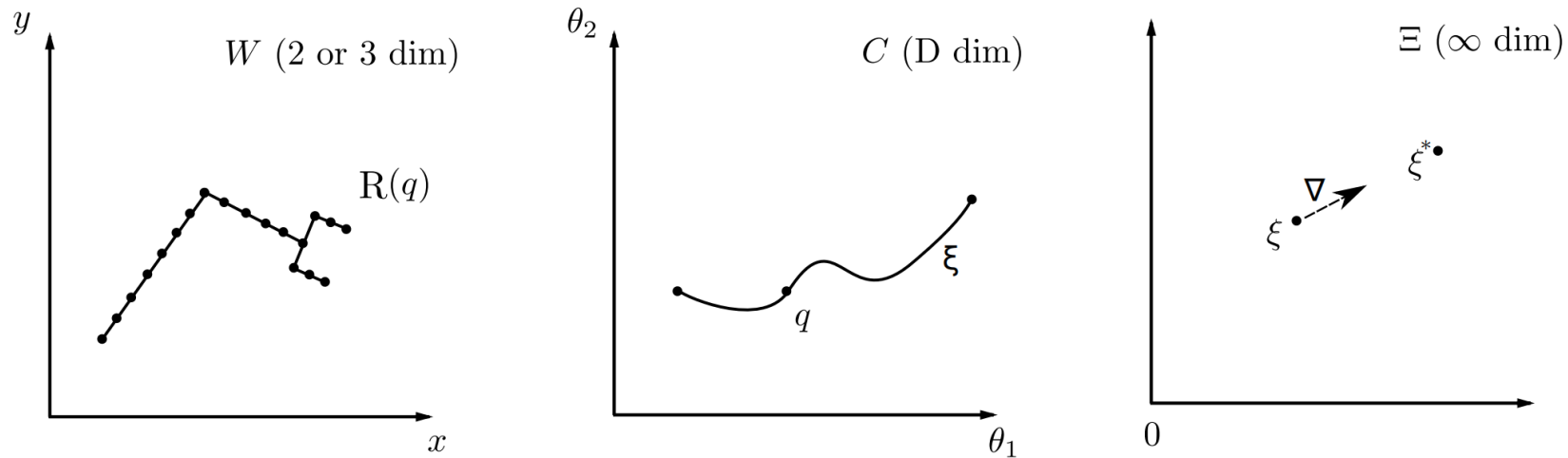
$$\xi^* = \operatorname{argmin}_{\{\xi \in \Xi\}} U[\xi]$$

s.t. $\xi(0) = q_s$

$$\xi(T) = q_g$$

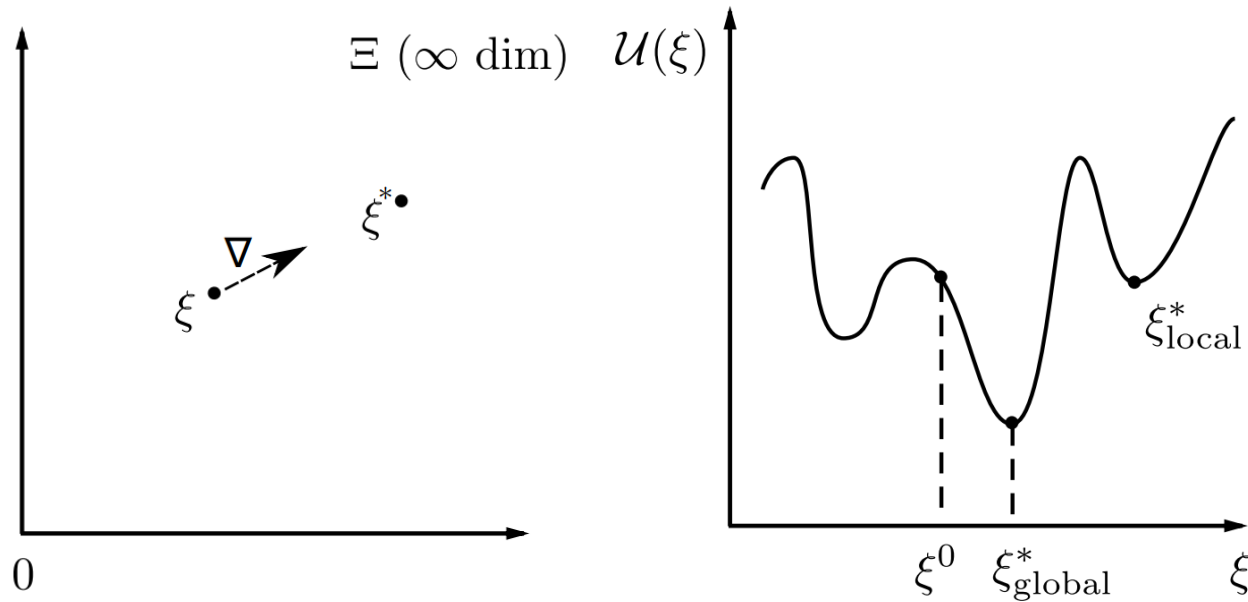
...(any other constraints we want)

Problem Specification: Optimization



- Want to optimize ξ to a global minimum of our objective U
⇒ Usually **too hard!**
- Instead, optimize ξ to a local minimum of our objective U
⇒ If the solution is bad, **resample ξ** and **try again**

Problem Specification: Optimization

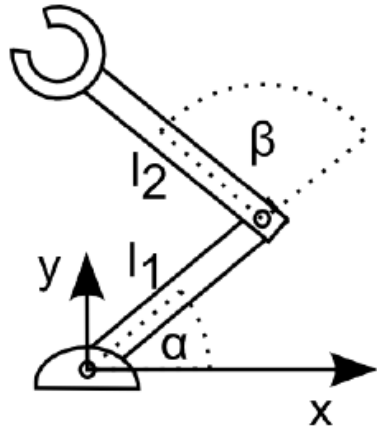


Points in the
direction of greatest
ascent for ξ

Gradient Descent is a popular
technique for finding local minima

$$\xi_{t+1} = \xi_t - \frac{1}{\alpha} \nabla_{\xi} U(\xi_t)$$

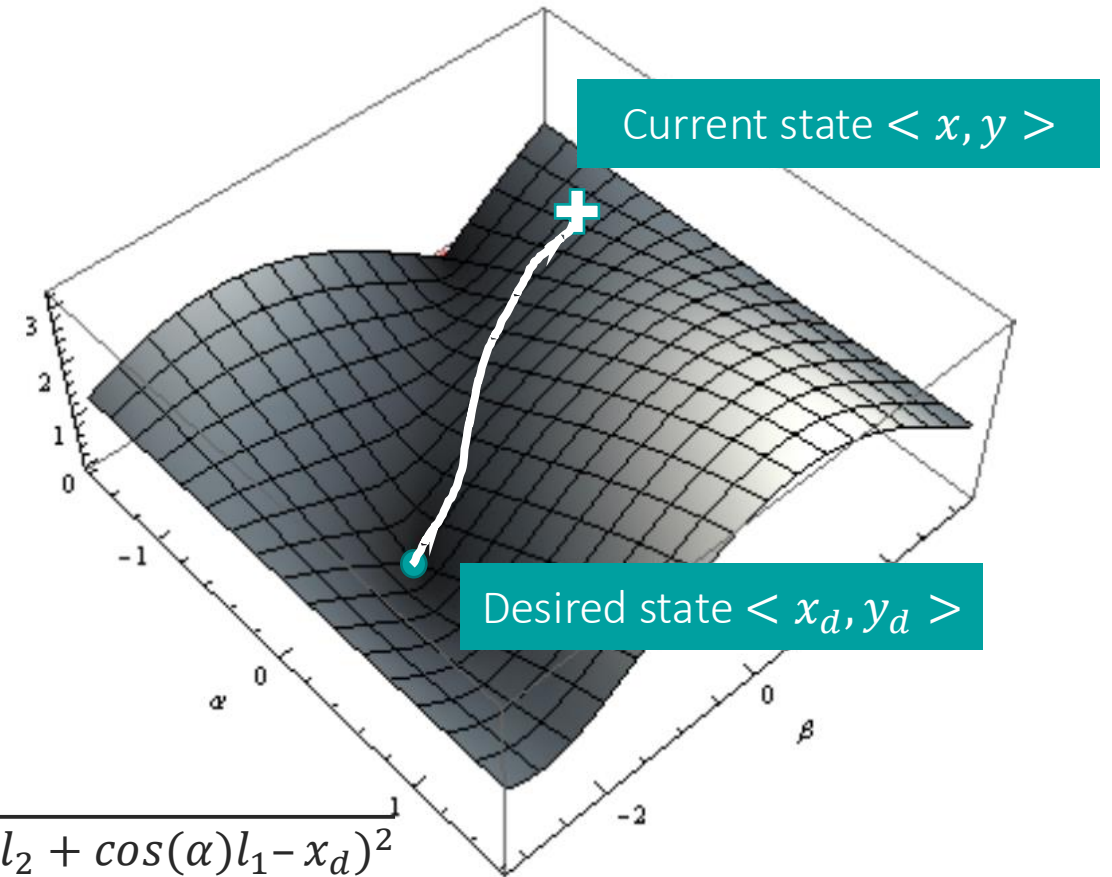
Recall: Motion Planning in EE Space



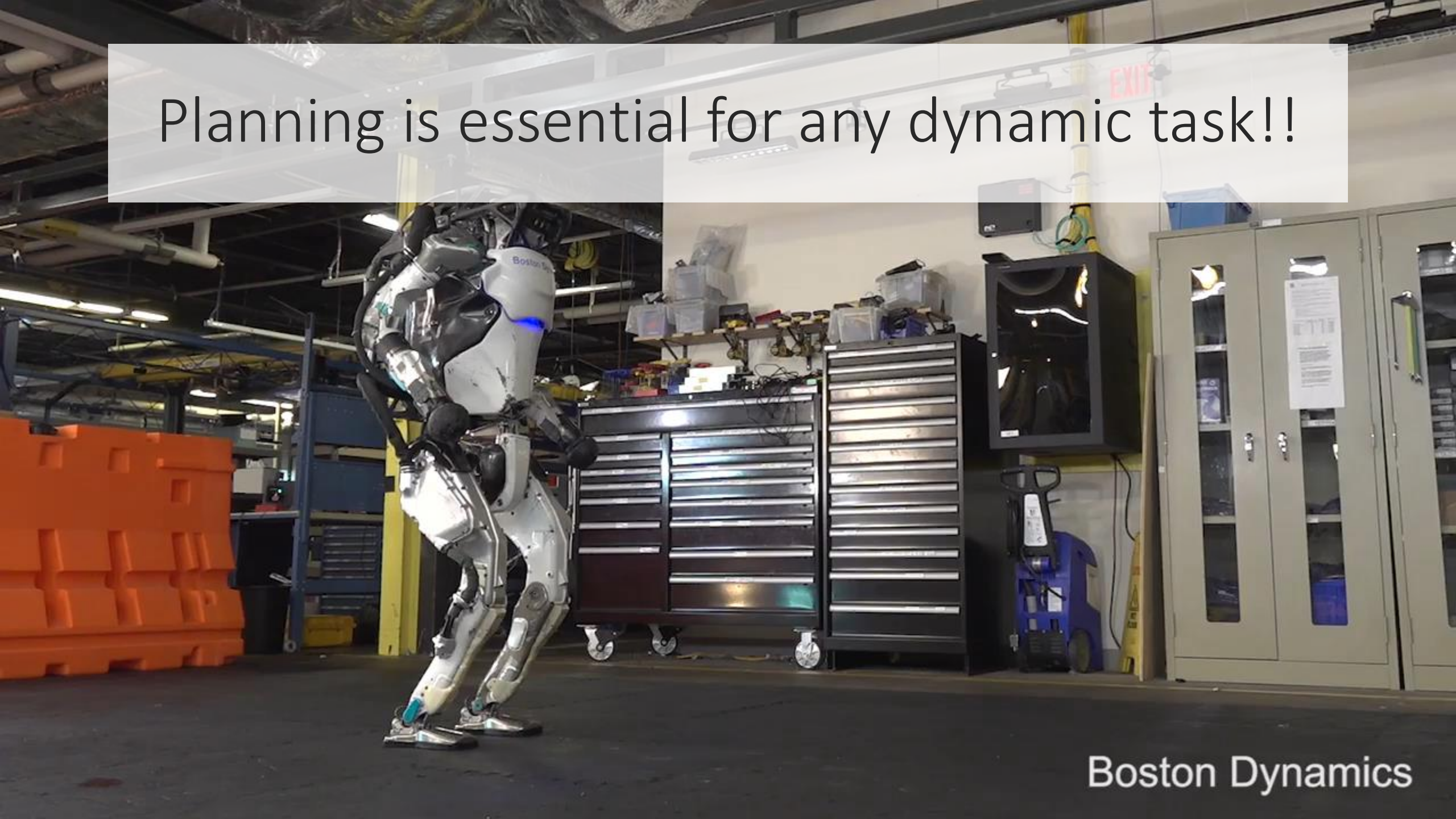
$$\begin{aligned}x &= l_1 \cos(\alpha) + l_2 \cos(\alpha + \beta) \\y &= l_1 \sin(\alpha) + l_2 \sin(\alpha + \beta)\end{aligned}$$

Just the Euclidean distance
between two vectors!

$$f_{x,y}(\alpha, \beta) = \sqrt{(\sin(\alpha + \beta)l_2 + \sin(\alpha)l_1 - y_d)^2 + (\cos(\alpha + \beta)l_2 + \cos(\alpha)l_1 - x_d)^2}$$

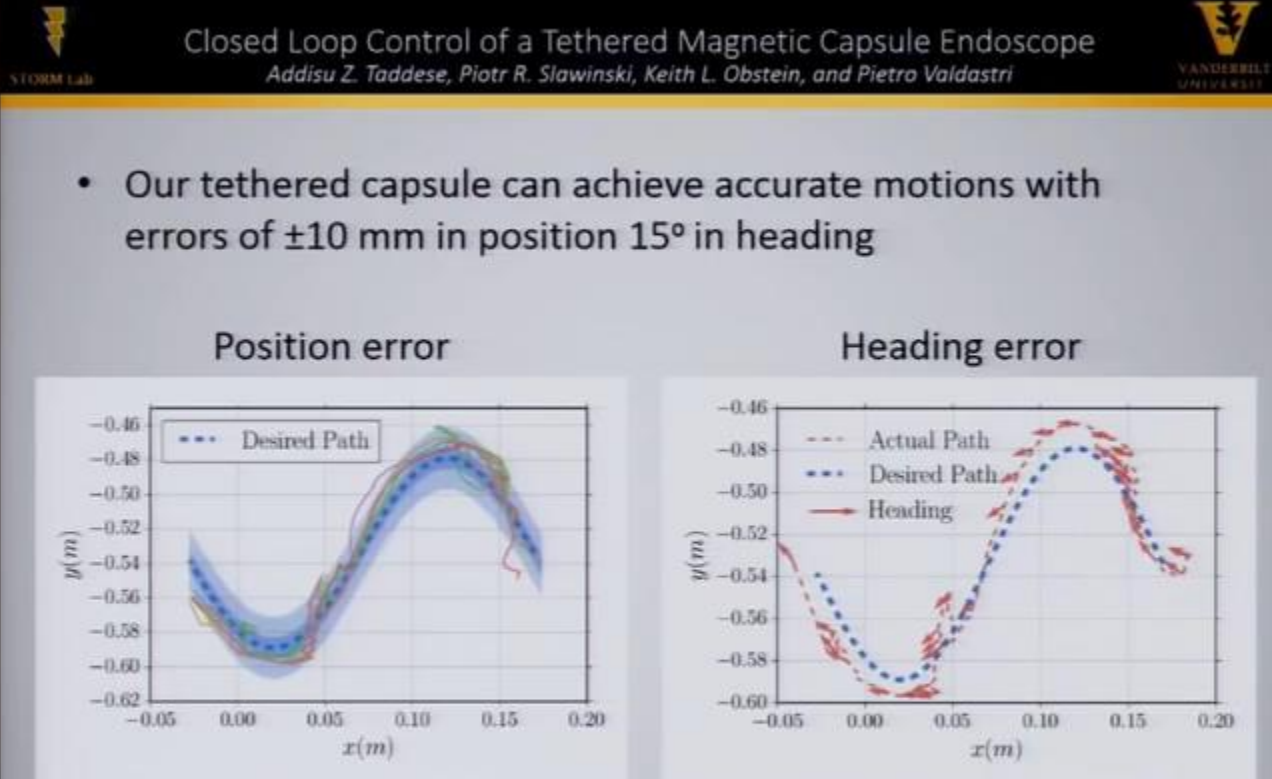
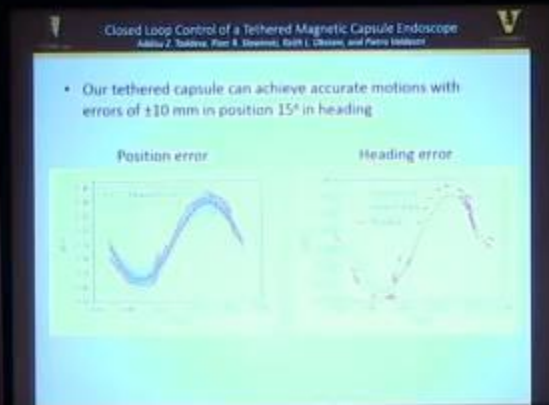


Planning is essential for any dynamic task!!

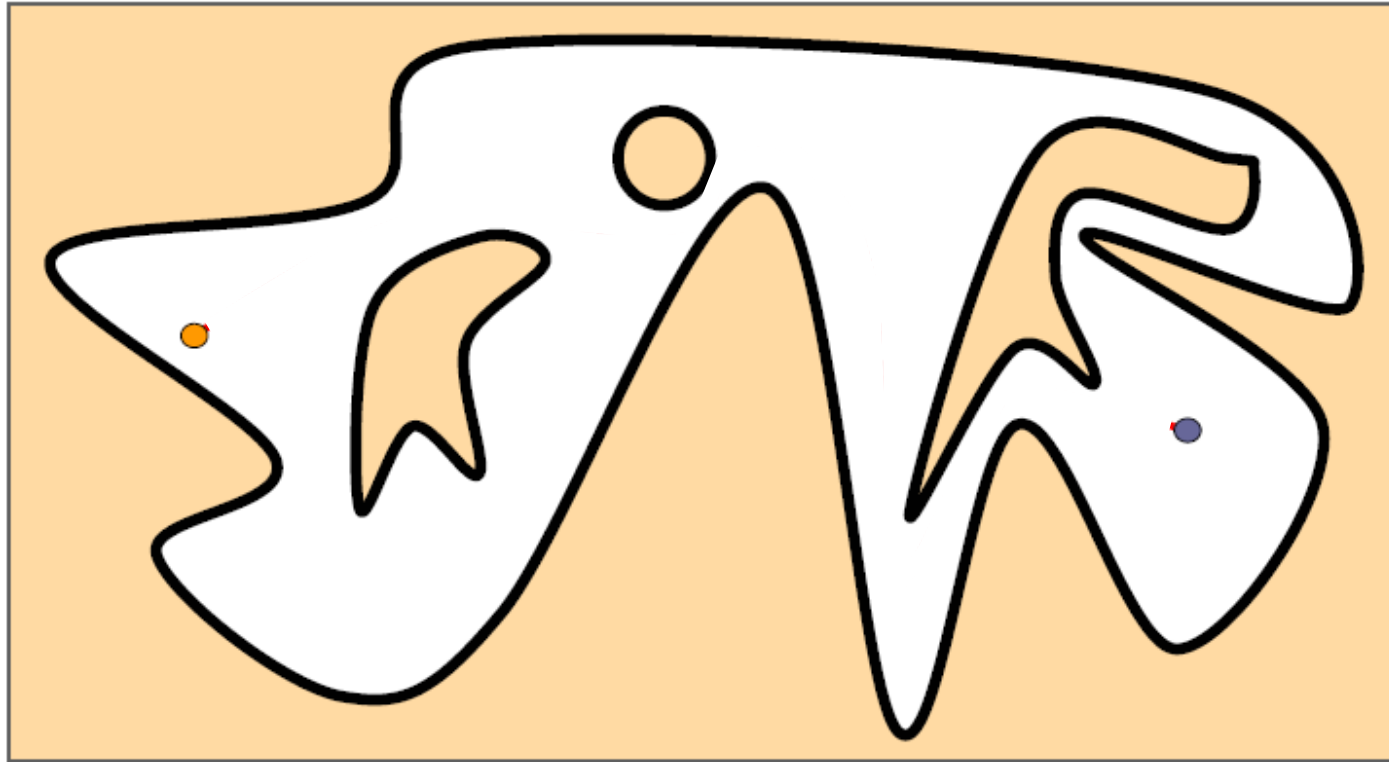


Boston Dynamics

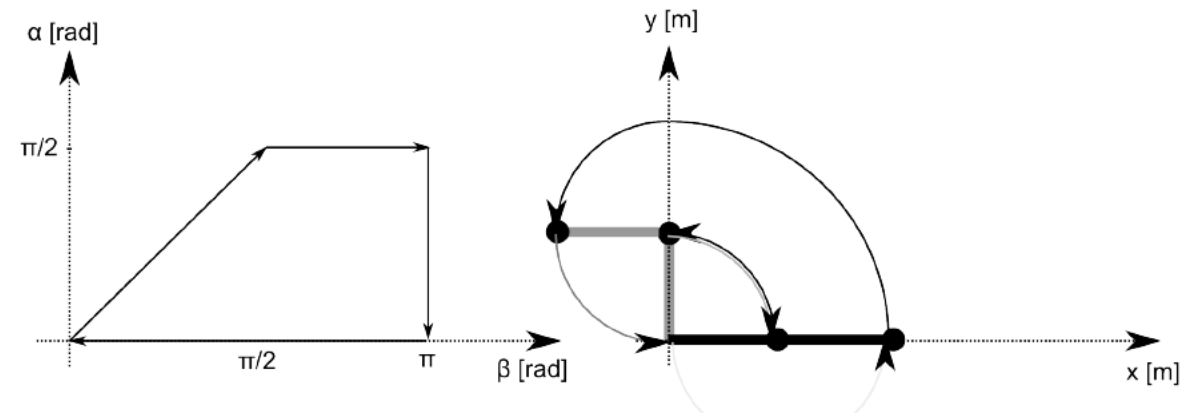
Motivating problem: Multi-robot coordination



Motivating Problem 2: Moving e-puck from start to goal state



Configuration Space



- Allows us to reduce robot positions to a single point
 - Very convenient for planning!
- One axis of configuration space for each degree of freedom of the robot
 - Convenient for planning as the space is the same as what the robot controls.
 - Removes the need to figure out how to get to a point in (X, Y)-space
- We still need to figure out how to put **obstacles** into configuration space!

Configuration Space

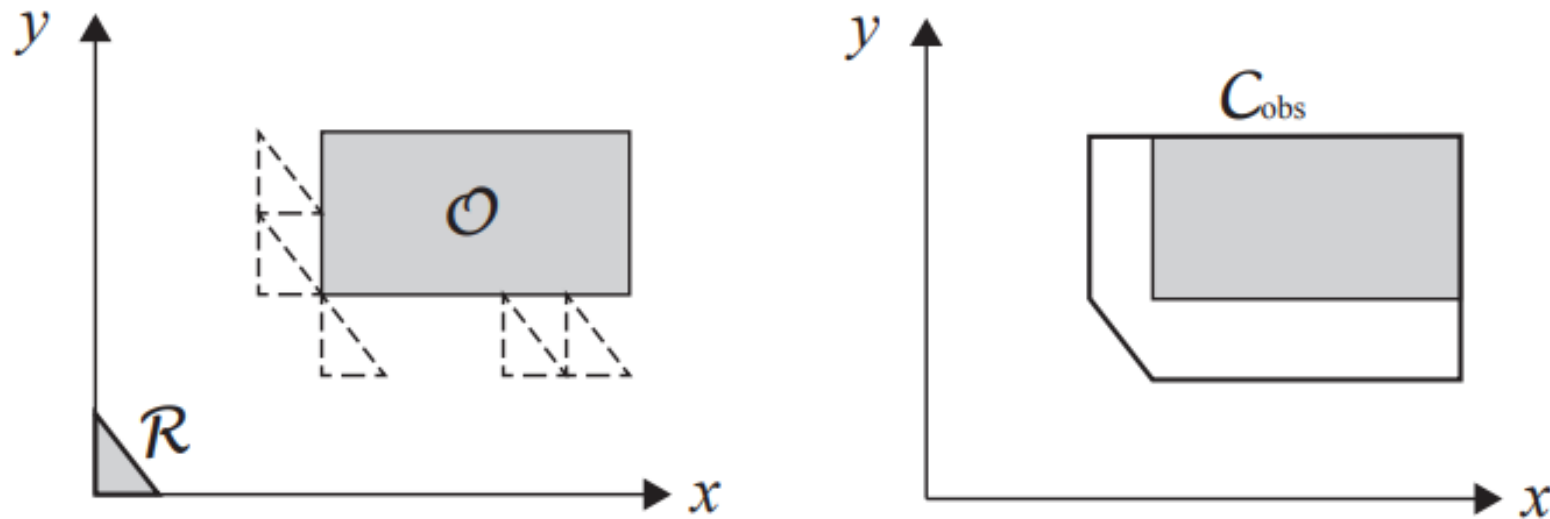
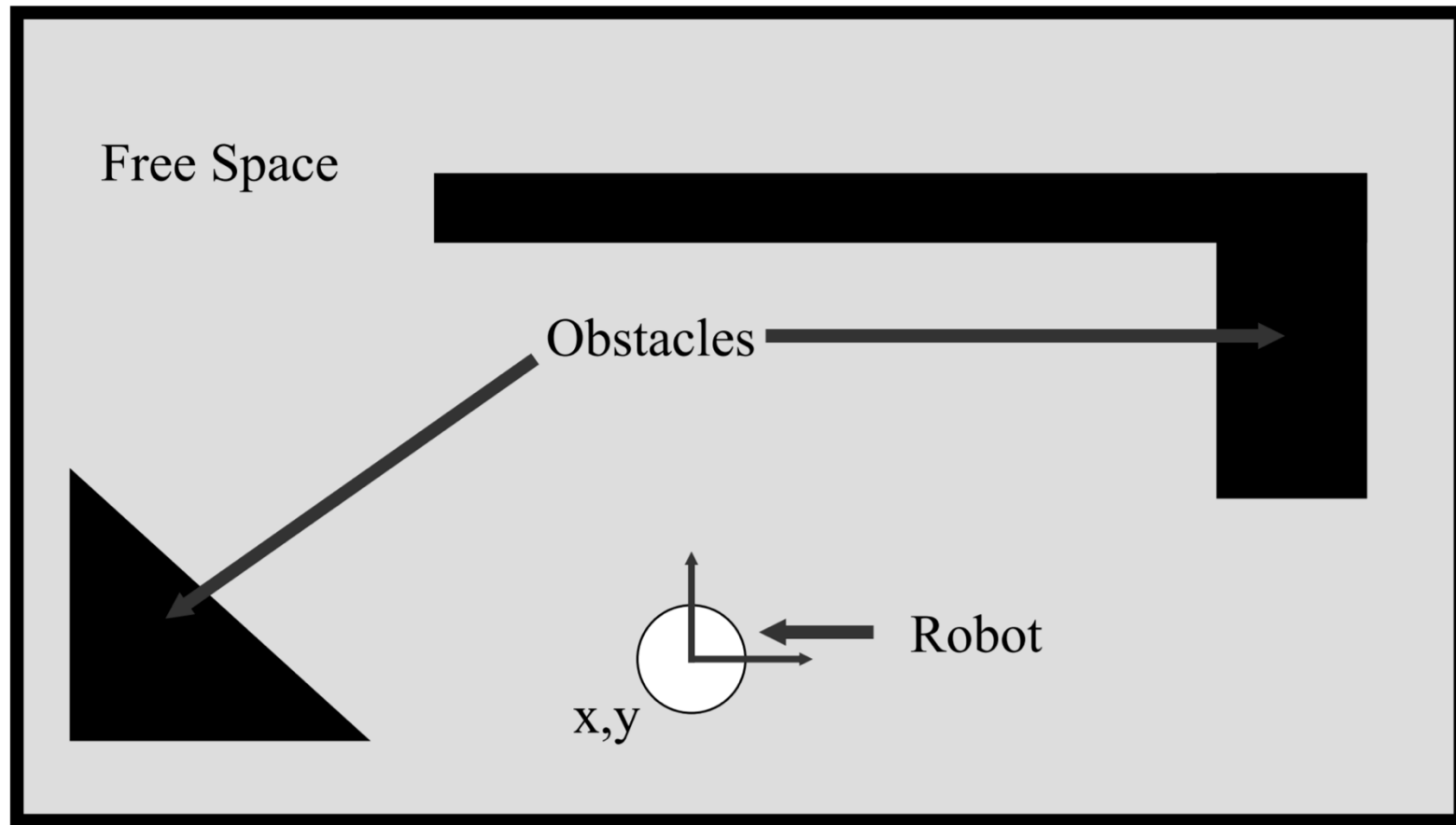


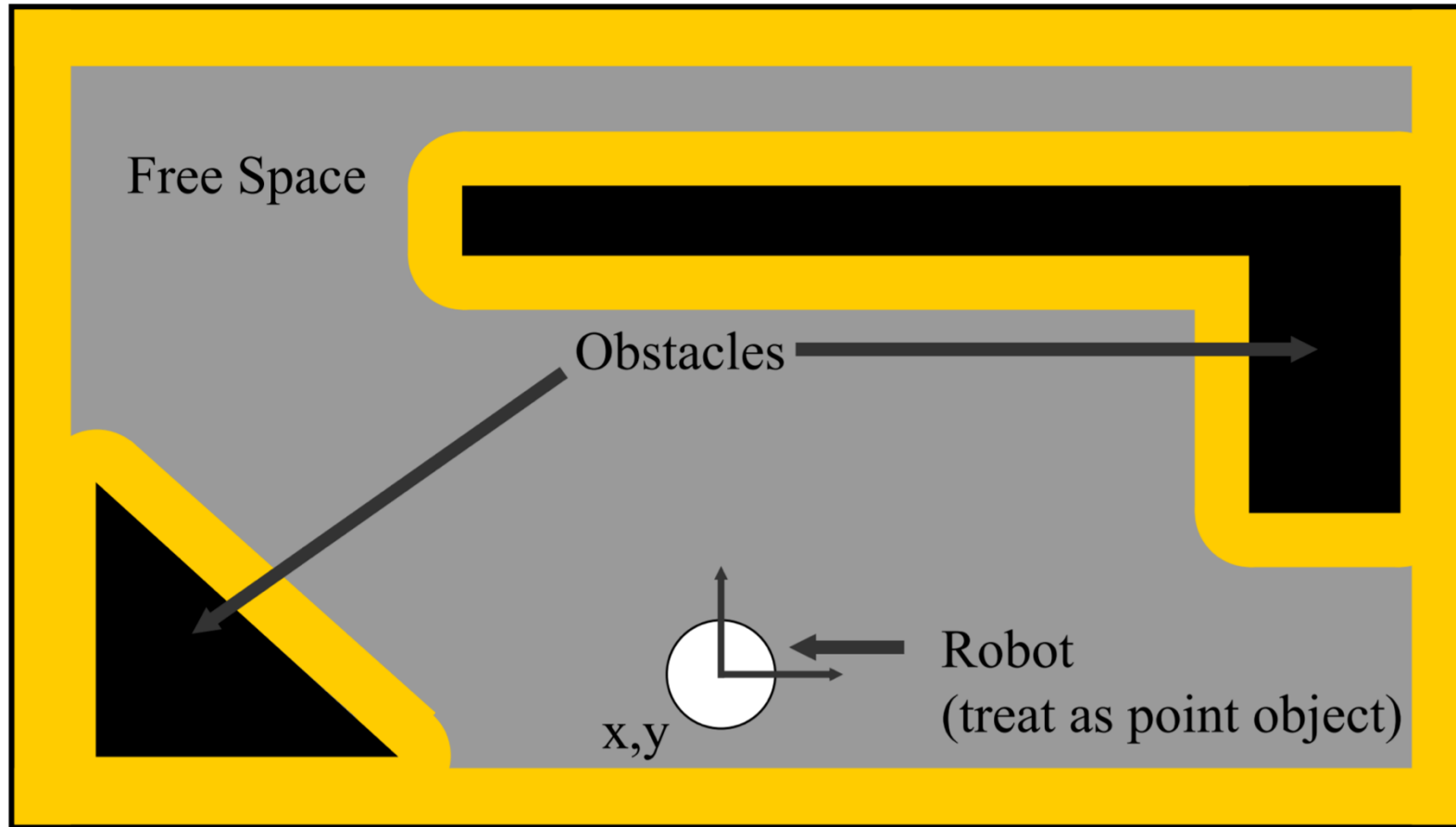
Figure 2.4: $\mathcal{C}_{obs}$ for a robot $\mathcal{R}$ that translates in x - y with a rectangle obstacle $\mathcal{O}$

Example of taking a robot that can translate across (X, Y) and an obstacle and putting them in configuration space.

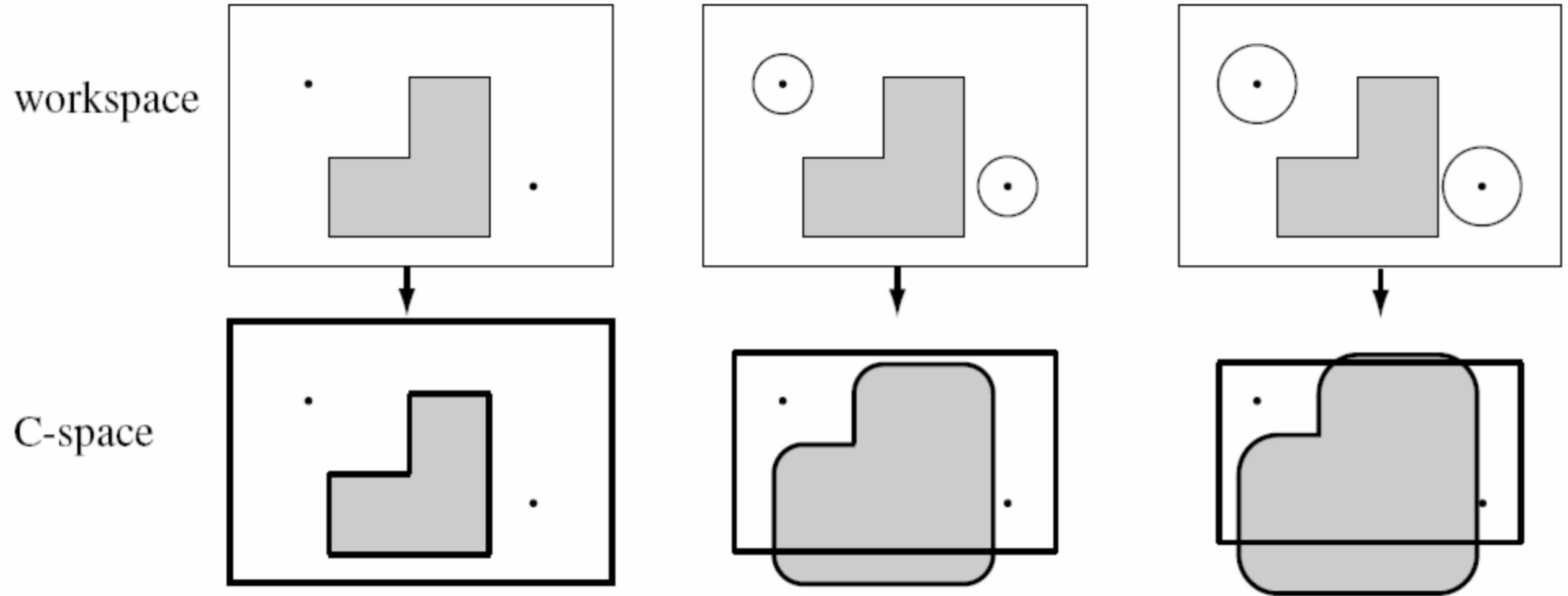
Configuration Space



Configuration Space



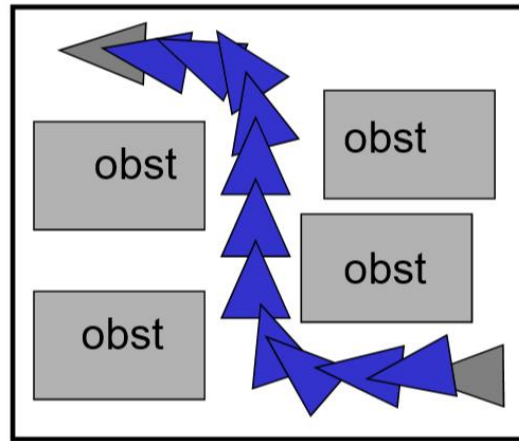
Configuration Space



Configuration Space

Workspace (x, y)

Each step has to check for collisions between the robot's body and obstacles!



▲ Robot

The robot's path is just a path through space – as a single point, collision checks are easy!

● Robot